Roboty mobilne

Instrukcja do zajęć laboratoryjnych

Ćwiczenie 2. Testowanie kodu w Pythonie z wykorzystaniem pytest

Wstęp

Instrukcja 2 rozwija zagadnienia z poprzedniego ćwiczenia. Tym razem nacisk należy położyć nie tylko na napisanie działających funkcji, ale przede wszystkim na przygotowanie testów automatycznych, które pozwolą sprawdzić poprawność obliczeń, obsługę błędów oraz zachowanie programu w przypadkach brzegowych. W ramach ćwiczenia należy wydzielić logikę programu do osobnych modułów tak, aby mogła być testowana bez połączenia z rzeczywistym urządzeniem.

W testowaniu należy odróżnić dwa podstawowe poziomy weryfikacji. Testy jednostkowe sprawdzają pojedynczą funkcję, metodę lub mały fragment logiki w izolacji. Testy integracyjne sprawdzają natomiast współpracę kilku elementów programu, na przykład generatora danych, komunikacji przez port szeregowy, konwersji odczytu oraz filtracji wyników.

We wstępie należy również zwrócić uwagę na składnię Pythona oraz podawanie typów. W Pythonie można stosować adnotacje typów, które zwiększają czytelność kodu i ułatwiają pracę w środowisku programistycznym, choć same w sobie nie wymuszają sprawdzenia typu w czasie wykonania programu.

```
def Sum(number_a: float, number_b: float) -> float:
    return number_a + number_b
```

W powyższym przykładzie zapis po dwukropku określa oczekiwany typ argumentów, natomiast zapis po strzałce -> wskazuje typ zwracanej wartości. Taki styl zapisu należy stosować również w funkcjach przygotowywanych podczas ćwiczenia, na przykład dla funkcji liczbowej, funkcji odczytu danych lub funkcji filtrującej.

Podstawowa składnia testu w bibliotece pytest opiera się na funkcjach rozpoczynających się od prefiksu test_ oraz na instrukcji assert. W praktyce najczęściej wykorzystywane będą: zwykłe porównania, sprawdzanie wyjątków, dekoratory przekazujące wiele zestawów danych oraz fixture przygotowujące obiekty wspólne dla wielu testów.

```
def test_sum():
    assert Sum(2.0, 3.0) == 5.0
```

```
import pytest
```

```
@pytest.mark.parametrize("value, expected", [(2, 4), (3, 9), (4, 16)])
def test_square(value, expected):
    assert value ** 2 == expected
```

```
with pytest.raises(ValueError):
    calculate_expression(0)
```

Na etapie przygotowania testów należy odpowiedzieć sobie na pytanie: co dokładnie ma zostać sprawdzone? W szczególności należy przetestować poprawne działanie dla typowych danych, przypadki brzegowe, niepoprawne wejścia oraz współpracę pomiędzy modułami. Taki sposób myślenia będzie obowiązywał w całej niniejszej instrukcji.

1. Opis biblioteki pytest oraz potrzeba stosowania testów jednostkowych oraz integracyjnych

Biblioteka pytest jest jednym z najczęściej stosowanych narzędzi do testowania w Pythonie. Pozwala ona uruchamiać testy w prosty sposób, grupować je w osobnych plikach, raportować błędy oraz wygodnie opisywać wiele przypadków testowych bez powielania kodu.

Należy przygotować krótkie omówienie różnicy pomiędzy testami jednostkowymi i integracyjnymi oraz wskazać, dlaczego oba rodzaje testów są potrzebne w projektach inżynierskich.

- Test jednostkowy sprawdza pojedynczy element logiki, na przykład funkcję calculate_expression(x), bez użycia rzeczywistego portu szeregowego i bez zależności od innych modułów.
- Test integracyjny sprawdza, czy kilka elementów działa razem poprawnie, na przykład: generator danych -> zapis do portu pozornego -> odczyt -> konwersja typu -> filtracja.
- Testy należy uruchamiać wielokrotnie w trakcie pracy nad kodem, a nie dopiero po zakończeniu całego projektu.
- Dobrze napisane testy powinny być krótkie, jednoznaczne i możliwe do uruchomienia bez sprzętu fizycznego.

W tej części należy także opisać podstawowe elementy składni pytest:

- funkcja testowa rozpoczynająca się od test_,
- instrukcja assert do sprawdzania wyniku,
- dekorator @pytest.mark.parametrize do wielokrotnego uruchamiania testu dla różnych danych,
- konstrukcja with pytest.raises(...) do sprawdzania wyjątków,
- fixture przygotowujący obiekty wykorzystywane w wielu testach.

2. Przygotowanie struktury folderów

W celu przygotowania odpowiedniej struktury należy stworzyć foldery src, tests oraz TestEnd2End. W folderze src powinien znaleźć się kod opracowany na poprzednich zajęciach. W folderze tests należy umieścić testy jednostkowe, natomiast w folderze testIntegration testy integracyjne lub scenariusze sprawdzające pełny przepływ danych.

Przykładowa struktura projektu może wyglądać następująco:

```
project/
├── src/
│   ├── math_utils.py
│   ├── serial_utils.py
│   ├── serial_fixture.py
│   ├── sensor_utils.py
│   └── filter_utils.py
├── tests/
│   ├── test_math_utils.py
│   ├── test_serial_utils.py
│   ├── test_sensor_utils.py
│   └── test_filter_utils.py
└── testIntegration /
    └── test_sensor_pipeline.py
```

Należy zainstalować w wirtualnym środowisku bibliotekę pytest za pomocą polecenia:

```
pip install pytest
```

Po przygotowaniu środowiska testy można uruchamiać przykładowo następującymi poleceniami:

```
pytest
pytest -v
pytest tests/test_math_utils.py -v
```

3. Przeprowadzenie testów funkcji liczbowej

Na podstawie opracowanej funkcji obliczającej wartość wyrażenia:

$$y = 2x^2 + \log(x)^2 + \sin(x)$$

należy wydzielić implementację do osobnego modułu, na przykład math_utils.py, a następnie przygotować do niej testy w pliku tests/test_math_utils.py.

Przykładowa definicja funkcji może mieć postać:

```
import math

def calculate_expression(x: float) -> float:
```

```
    if x <= 0:
        raise ValueError("Argument x musi być większy od 0.")
    return 2 * x**2 + math.log(x)**2 + math.sin(x)
```

W tej części należy przetestować co najmniej następujące elementy:

- poprawność obliczeń dla kilku dodatnich wartości x, obejmujących zarówno małe, jak i większe argumenty,
- porównanie wyniku z użyciem pytest.approx, ponieważ funkcja zwraca liczbę zmiennoprzecinkową,
- sprawdzenie, czy dla x = 0 zgłaszany jest wyjątek ValueError,
- sprawdzenie, czy dla x < 0 zgłaszany jest wyjątek ValueError.

W celu ograniczenia powielania kodu należy wykorzystać dekorator @pytest.mark.parametrize.

4. Zaimplementowanie klasy SerialPortFixture

Aby testować komunikację szeregową bez użycia rzeczywistego urządzenia, należy przygotować klasę pozornego portu szeregowego. W tej części należy pokazać przykład dziedziczenia klas, czyli utworzenia własnej klasy bazującej na klasie Serial.

```
from serial import Serial

class SerialPortFixture(Serial):
    def __init__(self):

    def write(self, data: bytes):
        pass

    def read(self, size: int = 1) -> bytes:
        pass
```

Zapis class SerialPortFixture(Serial): oznacza, że klasa SerialPortFixture dziedziczy po klasie Serial. Dzięki temu można zasymulować zachowanie portu szeregowego i nadpisać wybrane metody, na przykład write() oraz read().

W trakcie pracy w środowisku Visual Studio Code lub PyCharm typ klasy bazowej można podejrzeć przez przejście do definicji. W praktyce należy przytrzymać Ctrl i kliknąć lewym przyciskiem myszy na nazwie Serial. Środowisko powinno otworzyć definicję klasy lub odpowiedni plik biblioteki. Dodatkowo typ obiektu można sprawdzić programowo za pomocą funkcji type(...).

```
serial_port = SerialPortFixture()
print(type(serial_port))
print(isinstance(serial_port, Serial))
```

W zależności od systemu operacyjnego po przejściu do definicji można trafić do odpowiedniej implementacji biblioteki pySerial. Na komputerach z systemem Windows często będzie to implementacja związana z modułem serialwin32.

5. Przeprowadzenie testów poprawności wczytania i zapisu danych

Po przygotowaniu klasy SerialPortFixture należy napisać funkcje obsługujące zapis i odczyt danych, a następnie przetestować ich poprawność bez używania fizycznego portu COM.

Do przygotowania wspólnego obiektu testowego należy wykorzystać fixture biblioteki pytest. Taki fixture tworzy obiekt raz dla danego testu i przekazuje go jako argument funkcji testowej.

W tej części należy przetestować co najmniej:

- czy funkcja zapisu poprawnie koduje napis do typu bytes,
- czy dane zapisane do fixture mogą zostać następnie odczytane,
- czy wielokrotny zapis dopisuje kolejne dane do bufora,
- jak zachowuje się odczyt dla pustego bufora.

Przykładowe testy mogą wyglądać następująco:

```
import pytest
from serial_utils import SerialPortWrite, SerialPortRead

@pytest.mark.parametrize("message, expected", [("A", b"A"), ("Test",
b"Test")])
def test_serial_write(serial_port, message, expected):
    SerialPortWrite(serial_port, message)
    assert serial_port.serial_port_values == expected

def test_serial_write_then_read(serial_port):
    SerialPortWrite(serial_port, "Python")
    assert SerialPortRead(serial_port) == b"Python"
```

6. Przeprowadzenie testów generowania danych do czujnika oraz filtracji danych

W poprzednim ćwiczeniu dane czujnika były generowane sztucznie, a następnie filtrowane. W tej instrukcji należy przygotować do tych elementów osobne testy jednostkowe. Wskazane jest wydzielenie dwóch modułów: jednego odpowiedzialnego za generowanie odczytu z szumem oraz drugiego odpowiedzialnego za filtrację.

Przykładowa funkcja generująca odczyt może korzystać z biblioteki random i zwracać wartość odległości z losowym zakłóceniem. W przypadku takich funkcji należy pamiętać, że test nie powinien opierać się na przypadkowym wyniku bez kontroli. Należy albo ustawić ziarno generatora, albo sprawdzać, czy wynik mieści się w dopuszczalnym przedziale.

Dla generatora danych należy przetestować:

- czy zwracana wartość ma poprawny typ liczbowy,
- czy wynik mieści się w spodziewanym zakresie distance_cm ± noise_range,
- czy dla niepoprawnych parametrów zgłaszany jest wyjątek ValueError.

W przypadku filtracji można zastosować prostą średnią ruchomą. Wtedy test powinien porównywać wynik z wcześniej obliczoną oczekiwaną listą wartości.

```
def moving_average(data: list[float], window_size: int) -> list[float]:
    if window_size <= 0:
        raise ValueError("Rozmiar okna musi być dodatni.")
    if window_size > len(data):
        raise ValueError("Rozmiar okna nie może być większy od liczby
próbek.")
    return [sum(data[i:i + window_size]) / window_size for i in
range(len(data) - window_size + 1)]
```

Dla funkcji filtrującej należy przetestować:

- czy dla znanego wejścia zwracany jest dokładnie oczekiwany wynik,
- czy funkcja poprawnie reaguje na niepoprawny rozmiar okna,
- czy dane po filtracji są bardziej stabilne niż dane wejściowe, jeśli taka własność wynika z przyjętej implementacji.

7. Zaimplementowanie testów integracyjnych komunikacji z czujnikiem oraz filtracją danych

Ostatnim etapem ćwiczenia jest przygotowanie testu integracyjnego, który sprawdzi współpracę kilku wcześniej napisanych elementów. Taki test powinien korzystać z klasy SerialPortFixture zamiast z rzeczywistego sprzętu, ale jednocześnie przechodzić przez cały łańcuch przetwarzania danych.

Przykładowy scenariusz integracyjny może obejmować następujące kroki:

1. wygenerowanie przykładowego odczytu czujnika,
2. zamianę odczytu na napis i zapisanie go do pozornego portu szeregowego,
3. odczyt danych z portu, konwersję z bytes na str, a następnie na float,
4. dodanie kilku próbek do listy wejściowej i uruchomienie filtracji,
5. porównanie wyniku końcowego z oczekiwanym rezultatem lub z oczekiwanym zakresem wartości.

W tej części należy przetestować przede wszystkim to, czy moduły współpracują ze sobą poprawnie oraz czy błędy nie wynikają z niezgodności typów pomiędzy kolejnymi etapami przetwarzania danych.

Przykładowy test integracyjny może zostać zapisany w folderze TestIntegration i uruchamiany oddzielnie od testów jednostkowych.

```
def test_sensor_pipeline(serial_port):
    samples = [10.0, 10.5, 11.0]
    for sample in samples:
        SerialPortWrite(serial_port, f"{sample}")

    raw = SerialPortRead(serial_port)
    assert isinstance(raw, bytes)
```

Uwagi końcowe

Podczas oddawania ćwiczenia należy wyraźnie rozdzielić kod programu od kodu testów. Kod źródłowy powinien znajdować się w folderze **src**, natomiast testy w folderach **tests** oraz **testsIntegration**. Należy również pamiętać, że testy powinny być możliwe do uruchomienia bez rzeczywistego urządzenia, dlatego wszędzie tam, gdzie jest to możliwe, należy wykorzystywać obiekty pozorne, takie jak SerialPortFixture.

Za poprawnie wykonane ćwiczenie należy uznać takie rozwiązanie, w którym student potrafi: napisać funkcję z adnotacjami typów, przygotować testy jednostkowe i integracyjne, zastosować dekorator @pytest.mark.parametrize, sprawdzić wyjątki oraz wykorzystać fixture do testowania elementów komunikacji.